

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 2: Formatted Input, Selection, Git

Learning Outcomes

- How to capture user input in Java?
- How to do code selection in Java?
- What is version control and git?

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

Most interesting programs take **input**, perform some kind of interesting **process** or **computation** and produce **output**.

The programs we have seen so far only do computation and produce output.

Let's fix that...

Formatted Input

```
import java.util.Scanner;
```

```
Scanner input = new Scanner(System.in);  
System.out.println("How many weeks until the end of the semester?");  
int weeks = input.nextInt();  
int days = weeks * 7;  
System.out.println("In " + weeks + " weeks there are " + days + " days.");
```

`Scanner input = new Scanner(System.in);` uses Java's standard library to set up `input` to read from **standard input** — usually what you type into a program.

The function `input.nextInt()` waits for you to type in an integer and **returns** it.

Inputting Strings

```
Scanner input = new Scanner(System.in);
System.out.println("What is your name?");
String name = input.nextLine();
System.out.println("How old are you?");
int age = input.nextInt();
System.out.printf("Hello %s, you are %d years old.%n", name, age);
```

What happens if the program expects an `int`, but you enter a word?

Selection If-Then-Else

Selection If-Then-Else

So far, we have looked at straight-line programs.

Most interesting programs make some choice or decision based on the input.

```
Scanner input = new Scanner(System.in);
System.out.println("How old are you?");
int age = input.nextInt();

if (age < 18) {
    System.out.println("Let's watch Bambi at the cinema.");
} else {
    System.out.println("Let's watch Kill Bill at the cinema.");
}
System.out.println("It'll be fun.");
```

Breaking down If-Then-Else

```
if (age < 18) {  
    ...  
} else {  
    ...  
}
```

- **if** is a **keyword** — a word in the language with a fixed, special meaning.
- **(...)** : parentheses surround the condition.
 - **age < 18** is a **boolean expression** called the **condition**.
- **{...}** : **curly brackets** or **braces** surround the
 - **if block** or **true block**: statements only executed if the condition is true.
 - **else block** or **false block**. statements only executed if the condition is false.

Else is optional

```
Scanner input = new Scanner(System.in);
System.out.println("How old are you?");
int age = input.nextInt();

System.out.println("You could watch:");
System.out.println("Bambi");
if (age >= 12) {
    System.out.println("Spider-Man");
    System.out.println("X-Men");
}
if (age >= 18) {
    System.out.println("Kill Bill");
}
```

Else-If — choosing one of several options

```
Scanner input = new Scanner(System.in);
System.out.println("How old are you?");
int age = input.nextInt();

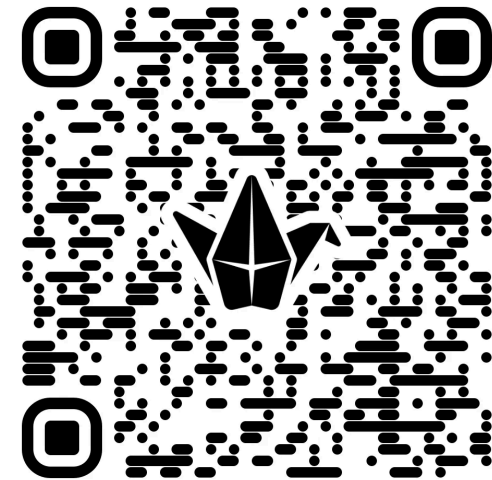
System.out.println("I think you would enjoy:");
if (age >= 18) {
    System.out.println("Kill Bill");
} else if (age >= 15) {
    System.out.println("Beverly Hills Cop");
} else if (age >= 12) {
    System.out.println("Spider-Man");
} else {
    System.out.println("Bambi");
}
```

Quiz 1 ([click here](#) or scan QRCode)

Which is the output of this program?

```
int age = 18;
if (age >= 18) {
    System.out.println("Adult");
} else if (age == 18) {
    System.out.println("Exactly 18");
} else {
    System.out.println("Younger than 18");
}
```

- a. Adult
- b. Exactly 18
- c. Younger than 18



Nested Ifs

An `if` statement is a statement. So you can put an `if` statement inside an `if` statement. This is called **nesting**.

```
if (age >= 18) {  
    if (time >= 21) {  
        System.out.println("Watch Kill Bill on TV by yourself.")  
    } else {  
        System.out.println("Let's watch Bambi on TV together.")  
    }  
} else {  
    if (time >= 21) {  
        System.out.println("Time for bed.");  
    } else {  
        System.out.println("Let's watch Bambi on TV together.")  
    }  
}
```

Whitespace and Comments

Whitespace Insensitivity and Indentation

Java, like many programming languages, is whitespace insensitive.

So I could do this:

```
Scanner input = new Scanner(System.in);System.out.println(
"How old are you?"); int age=input.nextInt();
    if (age <
18){System.out.println("Let's watch Bambi.");
}
else{
System.out.println("Let's watch Kill Bill at the cinema.");
}
System.out.println("It'll be fun.");
```

But don't do that. If you ask me to look at code like this, I'll tell you to format the spacing/indentation first.

Indentation

Use whitespace to make your code easier to read and understand.

When you use a **braced block**, **indent** its contents, say by 4 spaces from the left.

When you nest blocks, use multiple levels of indentation (8 spaces, 12 spaces...).

Put closing braces (`}`) on a line by themselves, one level left from block contents.

Some people prefer to use **tabs** instead of **spaces** for indentation. This is OK, but don't mix styles in the same project.

Historically, a tab was usually 8 spaces from the left. But this uses too much space when you have many levels of nesting. 4 or even 2 is usually preferred today.

Many text editors will insert spaces when you press tab, or turn all tabs into spaces if you want.

Comments

Comments are lines or blocks of text that are ignored by the computer. Use them to add remarks about what a program does.

```
// Ask the user's age.  
Scanner input = new Scanner(System.in);  
System.out.println("How old are you?");  
int age = input.nextInt();  
  
// Suggest a film to watch.  
if (age < 18) {  
    System.out.println("Let's watch Bambi at the cinema.");  
} else {  
    System.out.println("Let's watch Kill Bill at the cinema.");  
}
```

// starts a comment, which continues until the end of the line.

This will benefit you (or someone else) when reading your program later.

Bad Commenting 1

```
Scanner input = new Scanner(System.in);  
// Now I want you to put this message on the screen.  
System.out.println("How old are you?");  
int age = input.nextInt();
```

Reason: The computer is not sentient. It ignores your comments entirely. Comments are for your benefit, or the benefit of another programmer.

Bad Commenting 2

```
if (age < 18) {  
    // If age is less than 18, do this.  
    System.out.println("Let's watch Bambi at the cinema.");  
}
```

Reason: The comment just rephrases the code and adds no useful information.

Bad Commenting 3

```
Scanner input = new Scanner(System.in);
System.out.println("How old are you?");
int age = input.nextInt();

if (age < 18) {
    System.out.println("Let's watch Bambi at the cinema.");
} else {
    System.out.println("Let's watch Kill Bill at the cinema.");
}
```

Reason: No comments at all.

Bad Commenting 4

```
Scanner input = new Scanner(System.in);  
//System.out.print("How old are you?");  
System.out.println("How old are you?");  
int age = input.nextInt();  
// int age = input.nextLine();  
  
if (age < 18) {  
    System.out.println("Let's watch Bambi at the cinema.");  
}  
// else if (age >= 18) {else {  
    System.out.println("Let's watch Kill Bill at the cinema.");  
}
```

Reason: Sometimes comments are useful to disable a line of code temporarily or while trying to fix a bug but remember to clean them up afterwards.

Block Comments

You can make comments spanning multiple lines, starting `/*` and ending `*/`. This is mainly used to disable a block of code temporarily.

```
// Suggest a film to watch.  
if (age < 18) {  
    System.out.println("Let's watch Bambi at the cinema.");  
}  
// Removed violent film to avoid controversy.  
/*else {  
    System.out.println("Let's watch Kill Bill at the cinema.");  
}  
*/
```

If you accidentally move or delete the ending `*/`, you can inadvertently disable large parts of your program.

Booleans and Chars

Relational Operators

We have already seen:

- `age < 18` — less than: `age` is 17, 16, 15, ...
- `age >= 18` — greater than or equal: `age` is 18, 19, 20, ...

We also have:

- `age > 18` — greater than: `age` is 18, 19, 20, ...
- `age <= 18` — less than or equal: `age` is 18, 17, 16, ...

As well as:

- `age == 18` — equal: `age` is 18
- `age != 18` — not equal: `age` is ..., 15, 16, 17, 19, 20, 21, ...

Note `=` for assignment, but `==` for equality.

Char Type

A `char` variable holds a single character.

Character values are enclosed in single quotes (`' '`), not double quotes (`" "`).

```
char vogal1 = 'a';  
char vogal2 = 'e';  
char vogal3 = 'i';  
char vogal4 = 'o';  
char vogal5 = 'u';
```

```
char vogal1 = "a" ; // Compile Error
```

Inputting Char

```
Scanner input = new Scanner(System.in);  
System.out.println("Are you a student (y/n)");  
char name = input.nextLine().charAt(0);
```

Inputting Char

```
Scanner input = new Scanner(System.in);
System.out.println("Choose (r)ock, (p)aper or (s)ciissors.");
// Use .charAt(0) to get first character of a String.
int weapon = input.nextLine().charAt(0);

if (weapon == 'r') {
    System.out.println("You chose rock.");
} else if (weapon == 'p') {
    System.out.println("You chose paper.");
} else if (weapon == 's') {
    System.out.println("You chose scissors.");
} else {
    System.out.println("Cheater!");
}
```

Recap: Integer Values and Operators

Integers are values. You can assign them to variables and use operators to perform calculations with them:

```
int x = 5;  
int y = 10;  
int z = (x + y) * 2;
```

The arithmetic operators `+`, `-`, `*` and `/` take two numbers as input and produce one number as output.

Booleans (**true** and **false**) are also values with operations.

Boolean Operators

The logical operators `&&` (**logical AND**) and `||` (**logical OR**) take two Booleans as input and produce one boolean as output.

```
// Avoid nested if by using &&.
if ((age >= 18) && (time >= 21)) {
    System.out.println("Watch Kill Bill on TV by yourself.")
}
```

```
// Avoid code duplication by using ||.
if ((age < 0) || (age > 150)) {
    System.out.println("I don't believe you.")
}
```

Boolean Values

Booleans are values. You can assign them to variables and use operators to perform calculations with them:

```
// Note use of whitespace to align similar expressions for easy reading.  
boolean morning    = (time >= 6)  && (time < 12);  
boolean afternoon  = (time >= 12) && (time < 18);  
boolean daytime    = morning || afternoon;  
boolean nighttime  = !daytime;  
boolean partytime  = true;  
  
if (partytime) {  
    System.out.println("It's always party time!");  
}
```

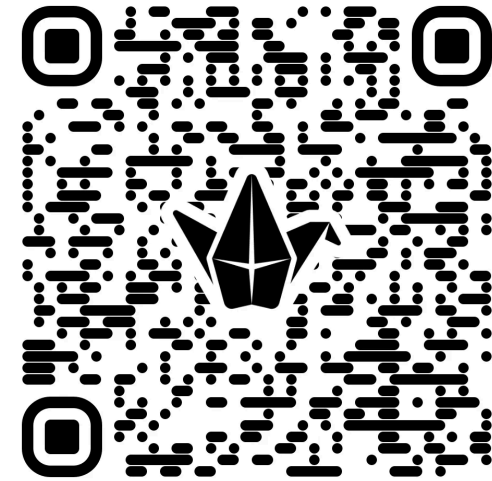
You can use **!** (logical NOT) to negate (flip) a boolean.

Quiz 2 ([click here](#) or scan QRCode)

Which are the values for `morning`, `afternoon`, `daytime` for the code below?

```
int hour = 20; // 8pm
boolean morning = (hour >= 6) && (hour < 12);
boolean afternoon = (hour >= 12) && (hour < 18);
boolean daytime = morning || afternoon;
```

- a. false, false, false
- b. true, false, true
- c. false, true, true
- d. true, true, true



Selection — Case-Switch

Case-Switch

Sometimes, we can avoid a complex sequence of `if` s using a `switch` statement.

```
Scanner input = new Scanner(System.in);
System.out.println("Choose (r)ock, (p)aper or (s)cissors.");
int weapon = input.nextLine().charAt(0);
switch(weapon) { // Switch picks what to do based on a variable's value.
    case 'r':
        System.out.println("You chose rock.");
        break;
    case 'p':
        System.out.println("You chose paper.");
        break;
    case 's':
        System.out.println("You chose scissors.");
        break;
    default: // If none of the cases match, execution continues here.
        System.out.println("Cheater!");
        break;
}
```

Case-Switch Pitfalls

You can omit the `break`, in which case execution continues to the next line:

```
switch(weapon) {  
    case 'r': case 'p': case 's':  
        System.out.println("You chose a weapon.");  
        break;  
    default:  
        System.out.println("Cheater!");  
        break;  
}
```

Case-Switch Pitfalls

But it is easy to forget `break` by accident, especially when editing code.

```
switch(weapon) {  
    case 'r': System.out.println("You chose rock."); // Oops.  
    case 'p': System.out.println("You chose paper.");  
    case 's': System.out.println("You chose scissors.");  
}
```

In practice, `switch` is rarely preferable to `if`, especially as the cases have to be constants (not expressions or conditions)... but there are some good uses.

Version Control and Git

Version Control

A source code many have multiple versions

- It may take several attempts to get a program right.
- The requirements for a program might change — new features may be needed.
- Several programmers may collaborate and share source code.

Informal ways to handle with this:

- Save multiple versions in different files: `Task1.java` , `Task2.java` , ...
- E-mail files to other programmers.
- Save files to a network shared drive.

A better way: Use **version control** (also known as source control, revision control).

The most popular source control system is **git**.

Git

A **repository** (or repo) stores all versions of all source code files of a project

Usually, there is a repository on server usually called **remote**

You can copy it copy onto your own **local machine** using the command

```
git clone REMOTE-REPOSITORY
```

Then you can edit your files locally



Git diff

When make a change, you see the changes using:

```
git diff
```



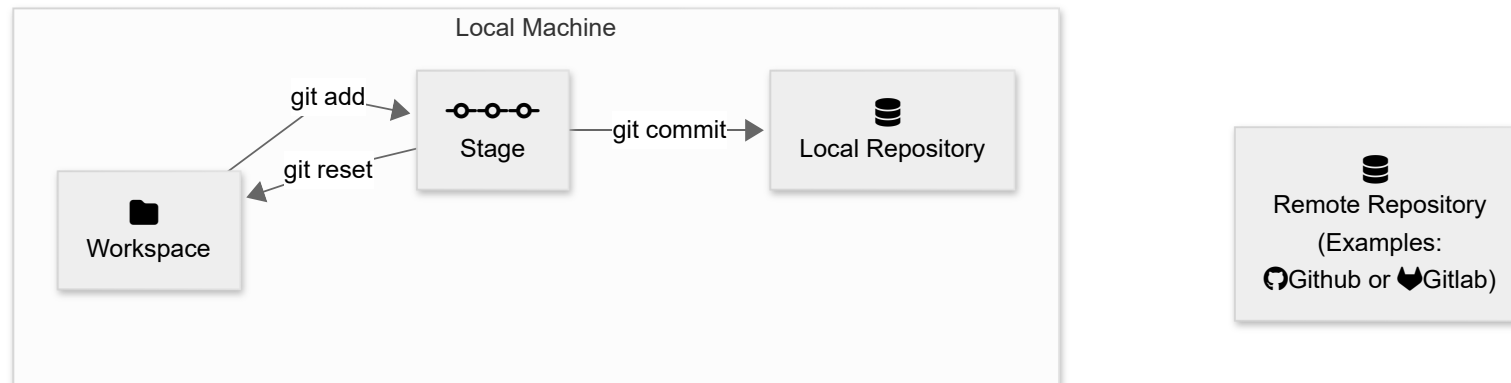
Git commit

When finish your changes, you have to `git add` them the files.

```
git add FILE
```

When finish adding, you `git commit` the changes to save them in the local repository by doing.

```
git commit -m "MESSAGE"
```



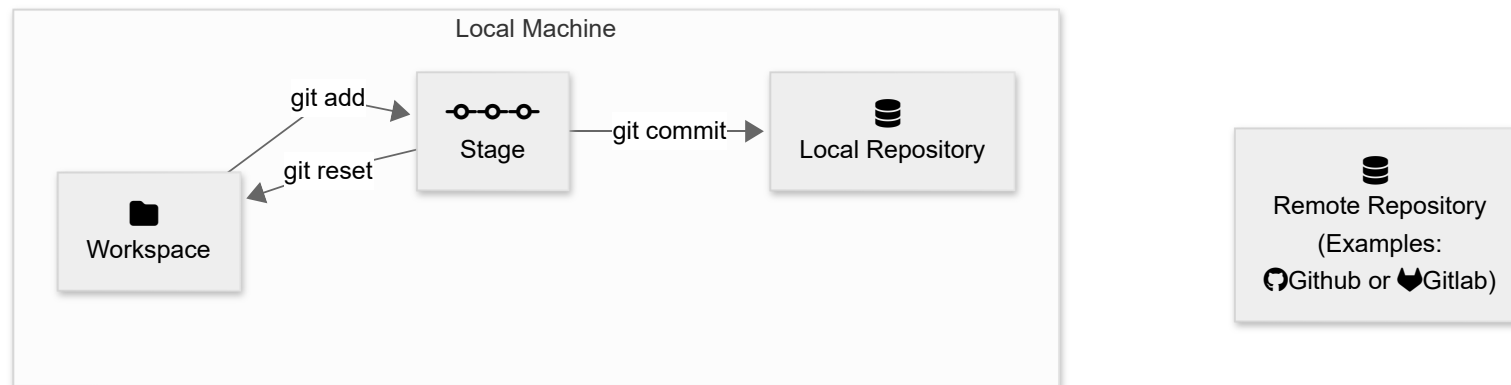
Git log

After make changes, you have to `git add` them first.

```
git add FILE
```

After add changes, you save changes in the local repository by:

```
git commit -m "MESSAGE"
```



Git push

After committed, to send commities to the server you should do:

```
git push
```



Git pull

To get the the latest version of the repository from the server

```
git pull
```

If you are working with another programmer who changes the same files as you, you may have to **merge** to resolve the differences. Sometimes this is automatic.

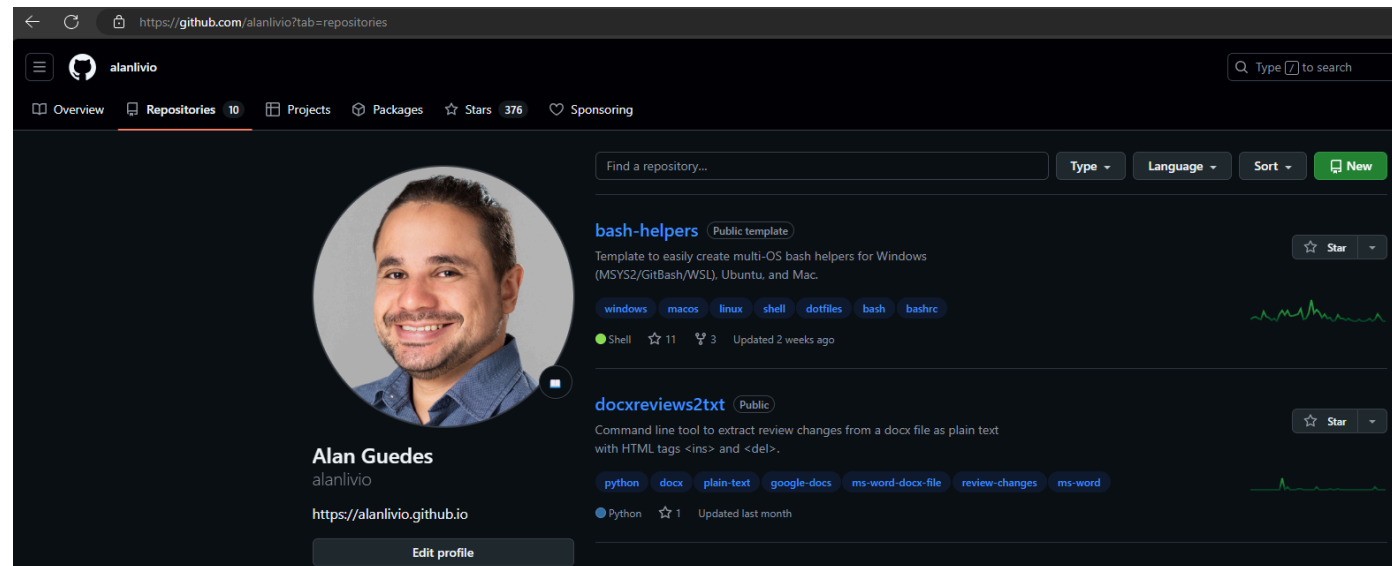


Git services: GitHub

Git is the name of the system and the program that implements it.

There are various services that will host Git repositories on a server.

Of these, **GitHub** is the best known. Anyone can create a free GitHub account.

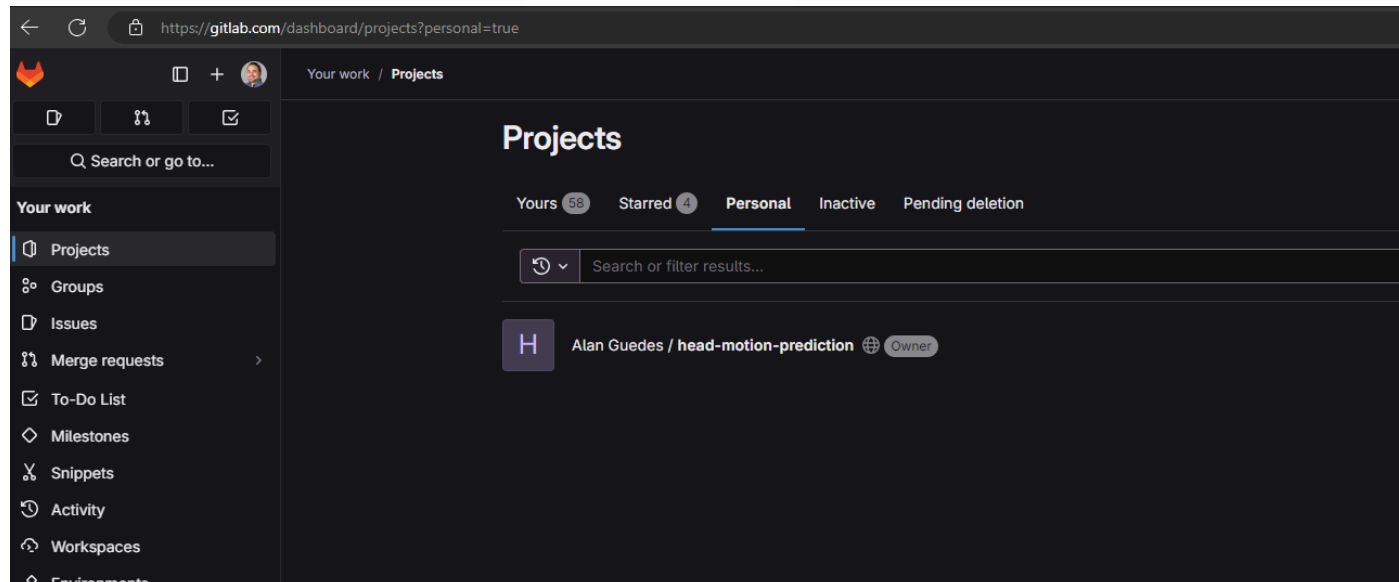


Git services: GitLab

GitLab is a company that develops software also called **GitLab** that lets anyone host their own (possibly private) service similar to **GitHub**.

The git difference from GitHub is that **GitLab** is based in open-source code.

GitLab the company also runs a GitLab installation (similar to GitHub) where anyone can create an account.

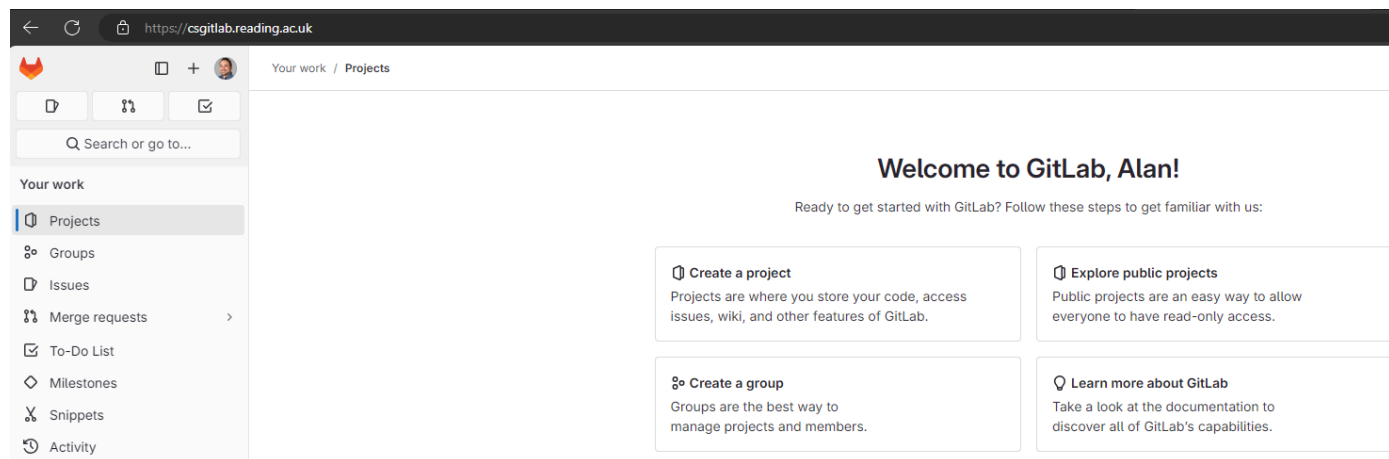


Git services at UoR: CSGitLab, ACT GitLab

Within the University, we have two GitLab installations:

- CSGitLab (<https://csgitlab.reading.ac.uk>): for Computer Science students specifically
- ACT Gitlab (<https://gitlab.act.reading.ac.uk>): run by the Academic Computing Team for any academic in the University

We will CSGitLab for CS1IP practicals and course work



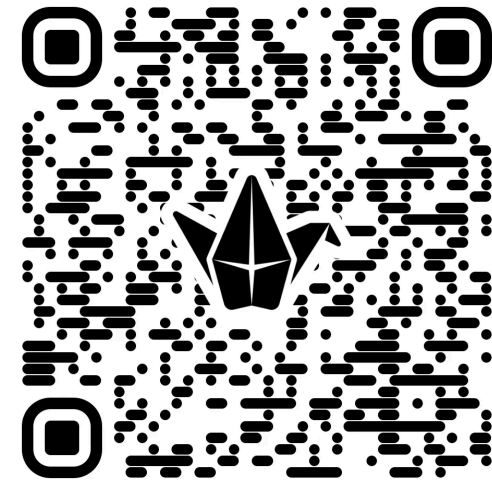
Demo: CSGitLab from command line

Demo: CSGitLab from VSCode

Quiz 3 ([click here](#) or scan QRCode)

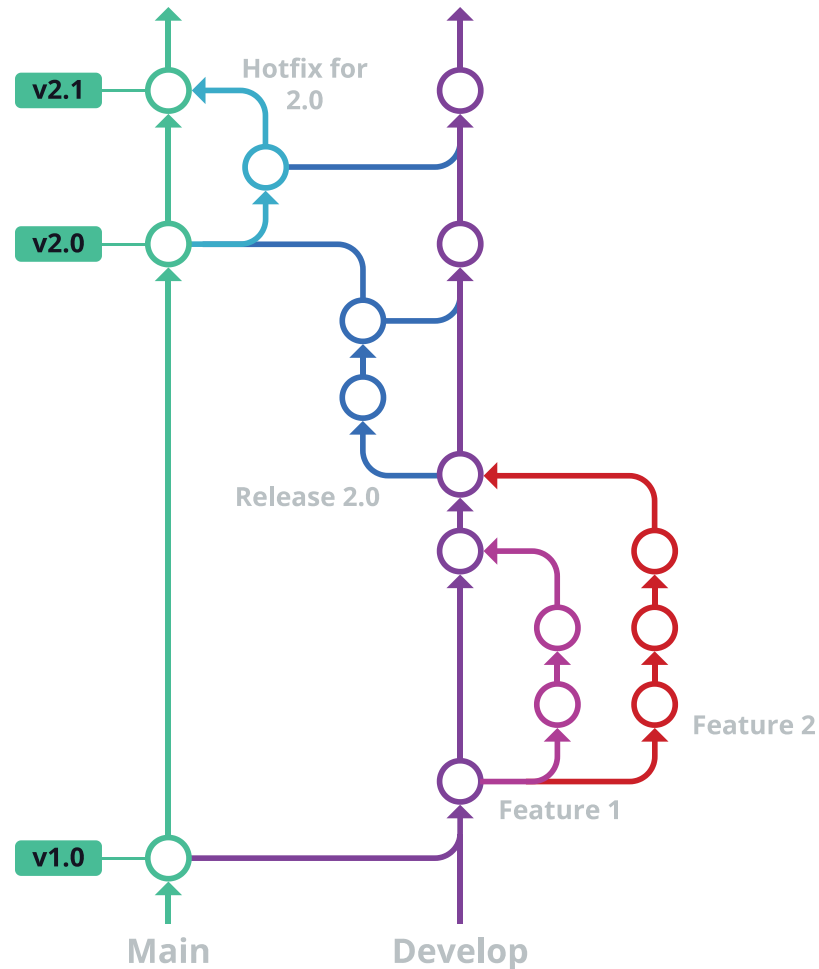
If you and another programmer both modify the same file in a Git repository and try to push your changes, what will happen?

- a. The latest push will always overwrite the previous one.
- b. Git will automatically merge all the changes without any conflicts.
- c. You may have to resolve a merge conflict if the same part of the file was changed.
- d. The other programmer's changes will be lost if your push happens second.



Git Flow

Larger projects use intensively git and follow the "Git Flow"



Review Learning Outcomes

Review Learning Outcomes

- How to capture user input in Java?

Using a Scanner

- How to do code selection in Java?

Using if, if-else, case-switch

- What is version control and git?

Version control allow us to maintain and evolve code. Git is the de-fact standard for doing it

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

Next time: Iteration